Лабораторная работа №3 «Разработка чек-листов и отчёта о тестировании». В документе составлены дымовой чек-лист и чек-лист критического пути для проекта DSLKit, показаны результаты их выполнения и оформлен отчёт о тестировании (Test Report) с метриками, покрытием требований и скриншотами.

1. Use Case и требования

Проверки чек-листов покрывают в том числе вариант использования UC-01 «Подать заявку в лист ожидания», который описан ниже. Он связан с требованием R4: проверки SM-07, CP-35 и CP-36; детальные тест-кейсы к нему приведены в лабораторной работе №2.

Название	**UC-01.** Подать заявку в лист ожидания (форма **«Записаться в бету»** на главной странице DSLKit)
Идентификатор, версия, приоритет	UC-01, версия 1.0, приоритет High: единственный способ оставить контакты для получения приглашения в бету.
Система и модуль	**Система:** *DSLKit* — веб-приложение «Редактор диаграмм как кода: слева текст, справа диаграмма», статический сайт без серверной части (каталог *site/*). **Модуль:** «Лист ожидания беты» — форма заявки на главной странице (раздел «Записаться в бету»). **Ссылка на модуль:** http://localhost:5173/#waitlist (запуск: *npm start*). **Ссылка на исходный код:** https://github.com/KsushaKhadzhinova/DSLKit-Project, файлы *site/index.html* (форма *#waitlist-form*) и *site/js/components/waitlist-form.js*.
Краткое описание цели и задачи	Посетитель оставляет имя, почту, нотацию диаграмм и размер команды, чтобы команда DSLKit прислала ему приглашение, когда откроется бета. Задача формы: принять только корректную заявку, сохранить её и показать страницу благодарности.
Основной актор	Посетитель сайта (неавторизованный). Внешних систем и сервера в сценарии нет: заявка остаётся в браузере посетителя.
Как перейти к форме	Форма находится на главной странице сайта в разделе **«Записаться в бету»** (якорь *#waitlist*), доступна всем посетителям **без регистрации и авторизации**. Перейти к ней можно так: **1.** Шапка главной страницы, кнопка **«В бету»** (en: «Join the beta»), *site/index.html*, строка 67. **2.** Первый экран, кнопка **«Записаться в бету»**, строка 88. **3.** Раздел «Нотации»: шесть ссылок **«Записаться в бету: UML / BPMN / ERD / DFD / IDEF0 / сети Петри»**, строки 127–172. **4.** Страницы *privacy.html* и *thanks.html*: кнопка **«В бету»** в шапке ведёт на *./#waitlist*. **5.** Прямой адрес: http://localhost:5173/#waitlist (русская версия), http://localhost:5173/en/#waitlist (английская). **6.** Клавиатура: Tab до кнопки или ссылки и Enter; ссылка-якорь

	прокручивает страницу к форме. **7.** React-приложение (*react-app*): кнопка **«Записаться в бету»** в шапке (UC-04, только сообщение, заявка не сохраняется).
Предусловия	1. Открыта главная страница: русская (/) или английская (/en/). 2. Раздел **«Записаться в бету»** показан, форма пуста: список **«Нотация»** стоит на пункте **«Выберите нотацию»**, в поле **«Размер команды»** значение 0, флажок согласия снят. 3. Скрипт *site/js/script.js* загружен (иначе форма не работает).
Основной поток событий	1. Посетитель переходит к форме одним из способов раздела «Как перейти к форме», например нажимает **«В бету»** (или **«Записаться в бету»**); страница прокручивается к разделу *#waitlist*. 2. Посетитель вводит имя в поле **«Имя*»** (подсказка **«От 2 до 60 букв. Допустимы пробел, дефис и апостроф.»**). 3. Посетитель вводит адрес в поле **«Электронная почта*»** (подсказка **«Адрес вида name@example.com. На него придёт приглашение.»**). 4. Посетитель выбирает нотацию в списке «Нотация*»: IDEF0, DFD, ERD, BPMN, UML или **«Сети Петри»**. 5. Посетитель вводит целое число в поле **«Размер команды*»** (подсказка **«Целое число от 0 до 500. Укажите 0, если работаете один.»**). 6. Посетитель при желании вводит текст в поле **«Комментарий (по желанию)»**. 7. Посетитель отмечает флажок **«Согласен получать письма о запуске беты и с политикой конфиденциальности*»**. 8. Посетитель нажимает кнопку **«Отправить»**. 9. Система считывает шесть полей и проверяет каждое по правилам П1–П6 из таблицы «Структура данных» (функция *validateWaitlist*). 10. Система формирует запись {name, email, notation, team, comment, createdAt}, где name, email и comment очищены от пробелов по краям, team приведён к числу, createdAt — время в формате ISO 8601; добавляет запись в массив *LocalStorage* по ключу *dslkit.waitlist* и пишет в консоль **«DSLKit: заявка сохранена в LocalStorage»** с названием нотации. 11. Система открывает страницу благодарности *thanks.html* (для английской версии en/*thanks.html*) с заголовком **«Спасибо, вы в списке беты»**. * — обязательное поле. В интерфейсе сайта звёздочек нет, обязательность указана фразой «Все поля, кроме комментария, обязательны».
Альтернативные потоки	**А1**. Одно или несколько полей не прошли проверку (шаг 9): под каждым неверным полем показывается красное сообщение (role=alert, например **«Имя не принято: поле пустое, короче двух символов или содержит цифры и знаки…»**), поле получает aria-invalid=true, фокус переходит на первое неверное поле в порядке имя, почта, нотация, команда, комментарий, согласие. **Заявка не сохраняется**, страница не меняется. Поле с ошибкой перепроверяется при каждом вводе, остальные при выходе из поля. После исправления сообщение скрывается. Сценарий продолжается

	с шага 8. **A2**. Согласие не отмечено (шаг 9): применяется A1; сообщение **«Без согласия мы не можем отправить приглашение. Поставьте галочку, чтобы продолжить.»** **A3**. Посетитель закрывает страницу или уходит со страницы до шага 8: данные не сохраняются, состояние системы не меняется. **A4**. Повторная заявка с тем же адресом (шаг 10): **принимается как новая отдельная запись**, сообщения о дубликате нет (дефект D-1). **A5**. Повторное нажатие **«Отправить»** после успешной отправки (шаг 11): игнорируется, вторая запись не создаётся (флаг submitting).
Постусловия	Успех: массив *dslkit.waitlist* в *LocalStorage* длиннее на одну запись, открыта страница *thanks.html* (en/*thanks.html*). Ошибка проверки (A1, A2): записей не добавилось, посетитель остаётся на главной странице, введённые значения остаются в полях.
Исключительные ситуации	**E1**. Запись в *LocalStorage* не удалась (хранилище переполнено или запрещено, шаг 10): метод *storage.set* возвращает false, но система всё равно открывает страницу благодарности; **сообщения об ошибке нет**, **заявка потеряна** (дефект D-2). **E2**. *LocalStorage* недоступен совсем (*getStorage* использует временное хранилище в памяти): заявка живёт только до закрытия страницы, посетитель видит страницу благодарности.
Связанные требования	**R4** — Форма заявки и валидация **R1** — Доступность страниц (главная, *thanks.html*, *privacy.html*) **R6** — Разметка и локализация (русская и английская версии) **R9** — Локальное хранилище
Подзадачи	**UC-01.1** — Показ формы и подсказок к полям **UC-01.2** — Чтение, очистка и проверка значений по П1–П6 **UC-01.3** — Показ ошибок, фокус и повторная проверка **UC-01.4** — Сохранение заявки в *LocalStorage* **UC-01.5** — Переход на страницу благодарности
Связанные задачи и use case	**UC-00** — Прочитать политику конфиденциальности (ссылка в тексте согласия, *privacy.html*) **UC-02** — Выбрать нотацию на карточке и перейти к записи (ссылки **«Записаться в бету: UML»**, **«…: BPMN»**) **UC-03** — Авторизация и вход в приложение: в текущей версии *не реализована*, форма и приложение (*site/app/*) работают без входа; задача на будущее, после появления серверной части **UC-04** — Запись в бету из React-приложения (*react-app/src/App.jsx*, лаб. 7) **D-1** — Повторные заявки принимаются **D-2** — Потеря заявки при сбое хранилища
Реализация	**IMPL-01** — *site/index.html*: разметка формы *#waitlist-form* **IMPL-02** — *site/js/components/waitlist-form.js*: *initWaitlistForm*, показ ошибок, отправка **IMPL-03** — *site/js/utils/validation.js*: *validateWaitlist* и правила П1–П6 **IMPL-04** — *site/js/utils/storage.js*: createStorage, *getStorage* **IMPL-05** — *site/thanks.html* и *site/en/thanks.html*: страница благодарности
Тест-кейсы и	**TC-WL-001** — Все поля корректны

проверки	**TC-WL-002…007** — Имя: границы и недопустимые символы (П1) **TC-WL-008…011** — Почта: формат и длина (П2) **TC-WL-012…017** — Размер команды: границы и тип (П4) **TC-WL-018** — Нотация вне списка (П3) **TC-WL-019, 020** — Комментарий: граница 500 (П5) **TC-WL-021** — Согласие не дано (П6) **TC-WL-022** — Пустая форма (А1) **SM-07** — Форма заявки есть на главной **CP-35** — Валидация корректной заявки **CP-36** — У полей есть подписи и сообщения об ошибках
Скриншоты интерфейса	**SCR-01** — Главная страница, русская версия (01-home.png) **SCR-02** — Главная страница, английская версия (02-home-en.png) **SCR-03** — Форма заявки: ошибки при пустой отправке (03-form-errors.png) **SCR-04** — Форма заявки: заполнена корректными данными (04-form-filled.png) **SCR-05** — Страница благодарности (05-form-submitted.png) **SCR-06** — Запись заявки в *LocalStorage* (12-uc-storage-record.png) **SCR-07** — Сбой записи: страница благодарности без ошибки (13-uc-storage-failure.png)
Ссылки	Страницы проекта (при запущенном npm start): http://localhost:5173/ — главная страница (русская версия) http://localhost:5173/#waitlist — раздел «Записаться в бету» http://localhost:5173/thanks.html — страница благодарности http://localhost:5173/privacy.html — политика конфиденциальности http://localhost:5173/en/ — главная страница (английская версия) http://localhost:5173/en/thanks.html — страница благодарности (английская версия) Репозиторий и источники: https://github.com/KsushaKhadzhinova/DSLKit-Project — репозиторий проекта https://svyatoslav.biz/software_testing_book/ — книга «Тестирование программного обеспечения. Базовый курс» https://github.com/KsushaKhadzhinova/report-gen — генератор отчётов

Структура данных заявки (сущность WaitlistEntry)

Поле	Тип и длина	Обязательность, уникальность	Маска, формат	По умолч.	Связь	Правило
name *	string (VARCHAR), длина 2–60	Да (NOT NULL); уникальность: Нет	^\p{L}[\p{L}' ’-]{1,59}$ (флаг u), перед проверкой пробелы по краям убираются	—	Связей нет	П1
email *	string (VARCHAR),	Да (NOT NULL); уникальнос	^[^\s@]+@[^\s@.]+(?:\.[^\s@.]+)*\.[^\s@.]{2,}$, пробелы по краям убираются	—	Адрес получателя приглашения;	П2

	длина 6–120	ть: Нет (дубликаты принимаются, D-1); рекомендуется уникальность без учёта регистра			связи с таблицами нет	
notation*	enum (6 значений), длина 3–5	Да (NOT NULL); уникальность: Нет	Точное совпадение с одним из: idef0, dfd, erd, bpmn, uml, petri (строчные)	не выбрано	FK → Notation.id (N:1), справочник NOTATIONS в validation.js	П3
team*	integer, длина 0–500 (до 3 цифр)	Да (NOT NULL); уникальность: Нет	Целое число; строка допустима при маске ^\d+$	0	Связей нет	П4
comment	string (TEXT), длина 0–500	Нет (допустима пустая строка); уникальность: Нет	Без маски; пробелы по краям убираются при сохранении	пусто	Связей нет	П5
consent*	boolean, длина —	Да, значение true; уникальность: —	Только true; в записи не хранится	false	→ документ «Политика конфиденциальности» (privacy.html)	П6
createdAt	string (ISO 8601, UTC), длина 24	Да (формирует система); уникальность: Нет (практически уникально)	ГГГГ-ММ-ДДТЧЧ:ММ:СС.ccccZ, Date.toISOString()	текущее время	Связей нет	—

Хранилище: массив JSON в LocalStorage браузера, ключ dslkit.waitlist (префикс dslkit. задаёт createStorage). Первичного ключа и индексов нет, запись определяется позицией в массиве. Связь «заявка → нотация» N:1 со справочником NOTATIONS из шести значений (site/js/utils/validation.js). * — обязательное поле. Поле consent проверяется, но в запись не попадает.

Данные формы: типы и допустимые значения

Поле	Тип данных	Обязательно	По умолчанию	Допустимые значения	Недопустимые значения
name* (#name)	string; input type=text	Да	пусто	От 2 до 60 символов; первая буква, далее буквы любого алфавита, пробел, апостроф, дефис; пробелы по краям игнорируются. Примеры: «Анна Иванова», «Ян», «O'Neil-Smith»	Пусто; 1 символ; 61 и более; цифры («Anna1»); знаки («Анна!»); первый символ не буква («-Анна»)
email* (#email)	string; input type=email	Да	пусто	Вид name@domain.zone, зона не короче 2 символов, до 120 символов. Пример: anna@example.com	Пусто; без @; без зоны («anna@example»); пробелы внутри; 121 символ и более
notation* (#notation)	enum; select, значение string	Да	не выбрано	Ровно одно из: idef0, dfd, erd, bpmn, uml, petri	Пусто (пункт «Выберите нотацию»); любое другое значение («xyz»); заглавные буквы («IDEF0»)
team* (#team)	integer; input type=number (в коде допускается и строка из цифр)	Да	0	Целое от 0 до 500 включительно. Примеры: 0, 5, 500, «12»	Меньше 0 («-1»); больше 500 («501»); дробное («2.5»); не число («abc»); пусто
comment (#comment)	string; textarea	Нет	пусто	От 0 до 500 символов (пусто допустимо)	501 символ и более (в браузере ввод ограничен maxlength=500)
consent* (#consent)	boolean; checkbox	Да	false (снят)	true (флажок отмечен)	false (флажок снят)
createdAt (формирует система)	string; время ISO 8601 (UTC)	Нет (не вводится)	текущее время в момент отправки	Строка вида 2026-10-25T12:00:00.000Z	Не вводится посетителем

Диаграммы процесса

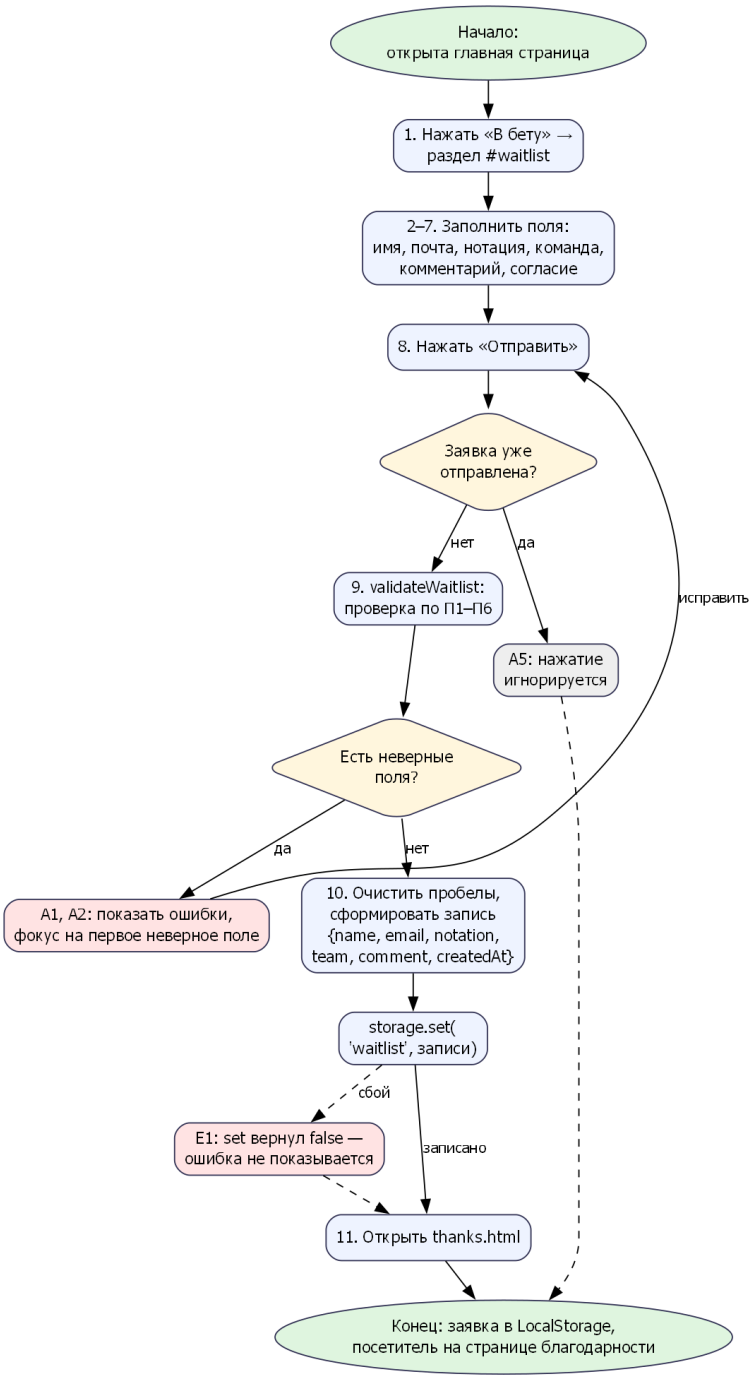


Рис. А. Блок-схема сценария UC-01: основной поток и альтернативные потоки

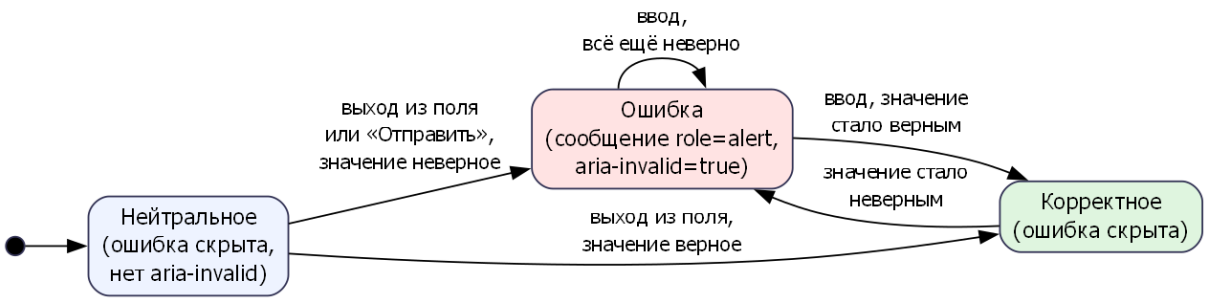


Рис. B. Состояния поля формы при проверке



Рис. C. Компоненты и обмен данными при отправке заявки

Модули и требования

ID	Требование
R1	Доступность страниц
R2	Загрузка стилей и скриптов
R3	Разбор DSL, преобразование в DOT и Mermaid
R4	Форма заявки и валидация
R5	Ошибки сервера: 404, 405, обход каталогов
R6	Разметка и локализация
R7	Безопасность: CSP и заголовки
R8	PWA, robots, sitemap
R9	Локальное хранилище

2. Дымовой чек-лист (Smoke)

ID	Треб.	Проверка	Статус	Комментарий
SM-01	R1	Главная страница открывается (200)	☑ Checked	
SM-02	R1	Редактор открывается (200)	☑ Checked	
SM-03	R1	Панель проектов открывается (200)	☑ Checked	
SM-04	R2	Стили главной загружаются	☑ Checked	
SM-05	R2	Скрипт главной загружается	☑ Checked	
SM-06	R3	Разбор DSL возвращает 2 блока и 1 стрелку	☑ Checked	
SM-07	R4	Форма заявки есть на главной	☑ Checked	
SM-08	R5	Несуществующий адрес даёт 404	☑ Checked	

3. Чек-лист критического пути (Critical Path)

ID	Треб.	Проверка	Статус	Комментарий
CP-01	R1	Страница / отдаётся как HTML	☑ Checked	
CP-02	R1	Страница /privacy.html отдаётся как HTML	☑ Checked	
CP-03	R1	Страница /thanks.html отдаётся как HTML	☑ Checked	
CP-04	R1	Страница /app/dashboard.html отдаётся как HTML	☑ Checked	
CP-05	R1	Страница /app/editor.html отдаётся как HTML	☑ Checked	
CP-06	R1	Страница /app/settings.html отдаётся как HTML	☑ Checked	
CP-07	R1	Страница /app/welcome.html отдаётся как HTML	☑ Checked	
CP-08	R1	Страница /en/ отдаётся как HTML	☑ Checked	
CP-09	R1	Страница /en/privacy.html отдаётся как HTML	☑ Checked	
CP-10	R1	Страница /en/thanks.html отдаётся как HTML	☑ Checked	
CP-11	R6	Главная: lang="ru", английская версия: lang="en"	☑ Checked	
CP-12	R6	На каждой странице ровно один h1	☑ Checked	
CP-13	R6	У главной есть canonical и alternate	☑ Checked	
CP-14	R6	У каждой страницы есть title и meta viewport	☑ Checked	
CP-15	R2	Все локальные ссылки на ресурсы главной открываются	☑ Checked	
CP-16	R7	Нет встроенных скриптов (CSP script-src self)	☑ Checked	
CP-17	R7	Заголовок Content-Security-Policy присутствует	☑ Checked	
CP-18	R7	X-Frame-Options = DENY	☑ Checked	
CP-19	R7	X-Content-Type-Options = nosniff	☑ Checked	
CP-20	R5	POST на статику даёт 405	☑ Checked	
CP-21	R5	Выход за корень (../) не отдаёт файлы	☑ Checked	
CP-22	R5	Страница 404 оформлена как HTML сайта	☑ Checked	
CP-23	R8	Манифест PWA: корректный JSON с иконками	☑ Checked	
CP-24	R8	Service worker доступен	☑ Checked	
CP-25	R8	robots.txt и sitemap.xml доступны	☑ Checked	
CP-26	R3	parseDsl: тип и заголовок диаграммы	☑ Checked	
CP-27	R3	parseDsl: порты блока разобраны	☑ Checked	
CP-28	R3	parseDsl: неизвестная строка попадает в skipped	☑ Checked	
CP-29	R3	parseDsl: пустой ввод не падает	☑ Checked	
CP-30	R3	toDot даёт digraph с обоими блоками	☑ Checked	
CP-31	R3	toMermaid возвращает непустой текст	☑ Checked	

CP-32	R3	sourceKey: один текст даёт один ключ, разные тексты разные	☑ Checked	
CP-33	R9	Хранилище: set/get возвращает значение	☑ Checked	
CP-34	R9	Хранилище: remove и значение по умолчанию	☑ Checked	
CP-35	R4	Валидация заявки: корректные данные дают пустой список ошибок	☑ Checked	
CP-36	R4	Форма: у полей есть label и сообщения об ошибках	☑ Checked	

Отчёт о тестировании (Test Report)

4. Общая информация

Параметр	Значение
Проект	DSLKit (site/), ветка testing-course
Дата	25.10.2026
Среда	Windows 11, Node.js, локальный сервер tools/server.js; браузер Microsoft Edge (скриншоты)
Способ	Автоматические проверки (npm run tc:lab3) и ручная проверка в браузере

5. Объём тестирования

Тестировалось: доступность страниц (RU и EN), ресурсы, DSL-парсер и экспорт в DOT и Mermaid, форма заявки, ошибки сервера, защитные заголовки, PWA, хранилище. Не тестировалось: отрисовка диаграмм внешним сервисом Kroki, кроссбраузерность, нагрузка.

6. Результаты

Статус	Количество
Passed	44
Failed	0
Blocked	0
Всего	44

Метрики

Метрика	Формула	Значение
Процент выполненных	Выполненные / Всего × 100%	100%
Процент успешных	Passed / Выполненные × 100%	100%
Плотность дефектов	Дефекты / Количество тестов	0
Покрытие требований	Покрытые / Всего × 100%	100% (9 из 9)

7. Обнаруженные дефекты

Проваленных проверок нет, баг-репорты не создавались.

Примечание: при отладке проверка SM-06 падала из-за ошибки в синтаксисе тестовых данных (DSL требует diagram, box … {, arrow). Это дефект теста, он исправлен, к продукту отношения не имеет.

8. Покрытие требований

Треб.	Описание	Проверки	Результат

R1	Доступность страниц	13: SM-01 … CP-10	Passed
R2	Загрузка стилей и скриптов	3: SM-04 … CP-15	Passed
R3	Разбор DSL, преобразование в DOT и Mermaid	8: SM-06 … CP-32	Passed
R4	Форма заявки и валидация	3: SM-07 … CP-36	Passed
R5	Ошибки сервера: 404, 405, обход каталогов	4: SM-08 … CP-22	Passed
R6	Разметка и локализация	4: CP-11 … CP-14	Passed
R7	Безопасность: CSP и заголовки	4: CP-16 … CP-19	Passed
R8	PWA, robots, sitemap	3: CP-23 … CP-25	Passed
R9	Локальное хранилище	2: CP-33 … CP-34	Passed

9. Скриншоты

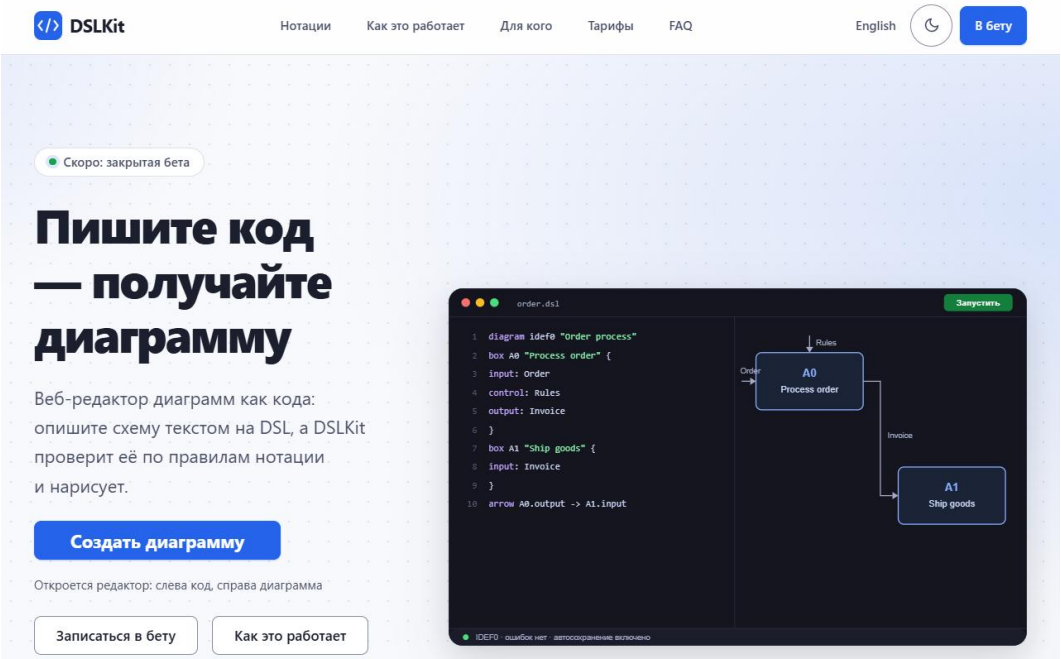


Рис. 1. R1: главная страница (RU)

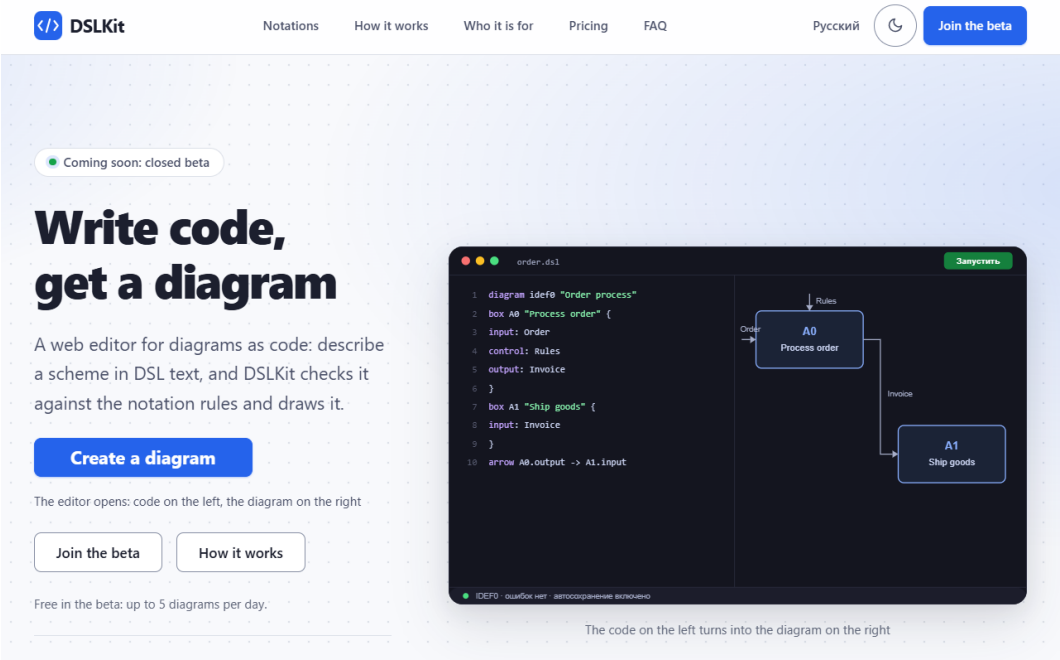


Рис. 2. R1, R6: английская версия

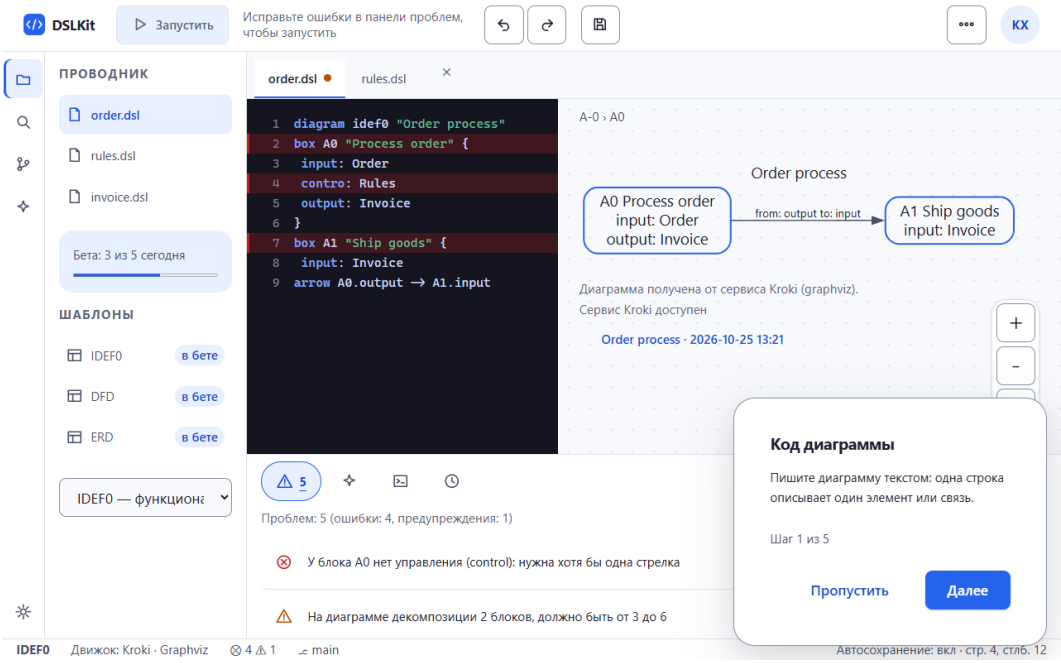


Рис. 3. R1: редактор

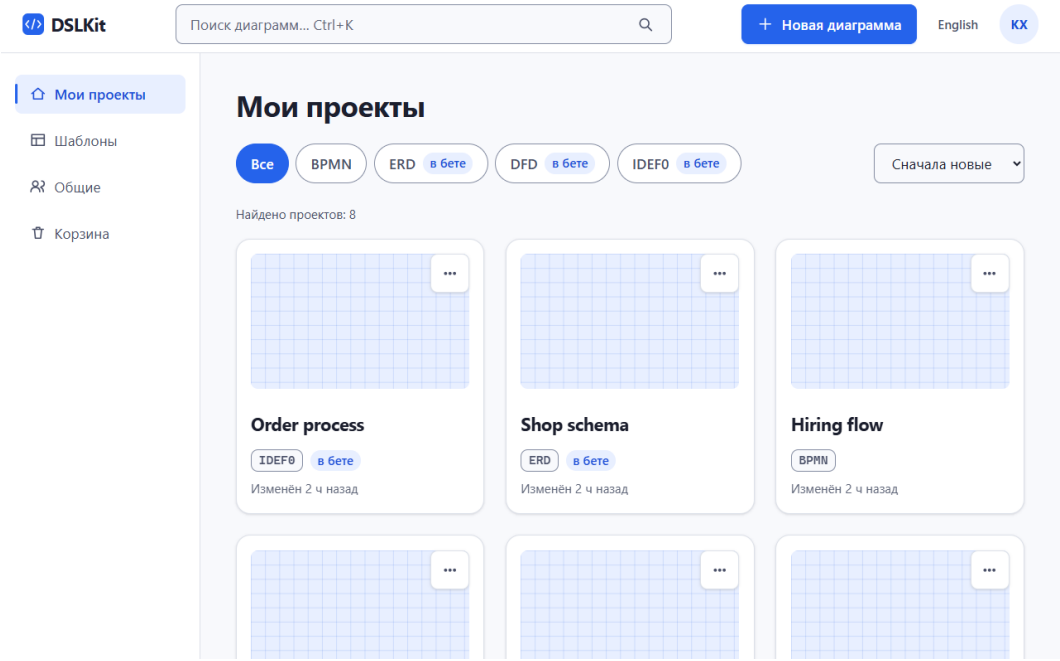


Рис. 4. R1: панель проектов

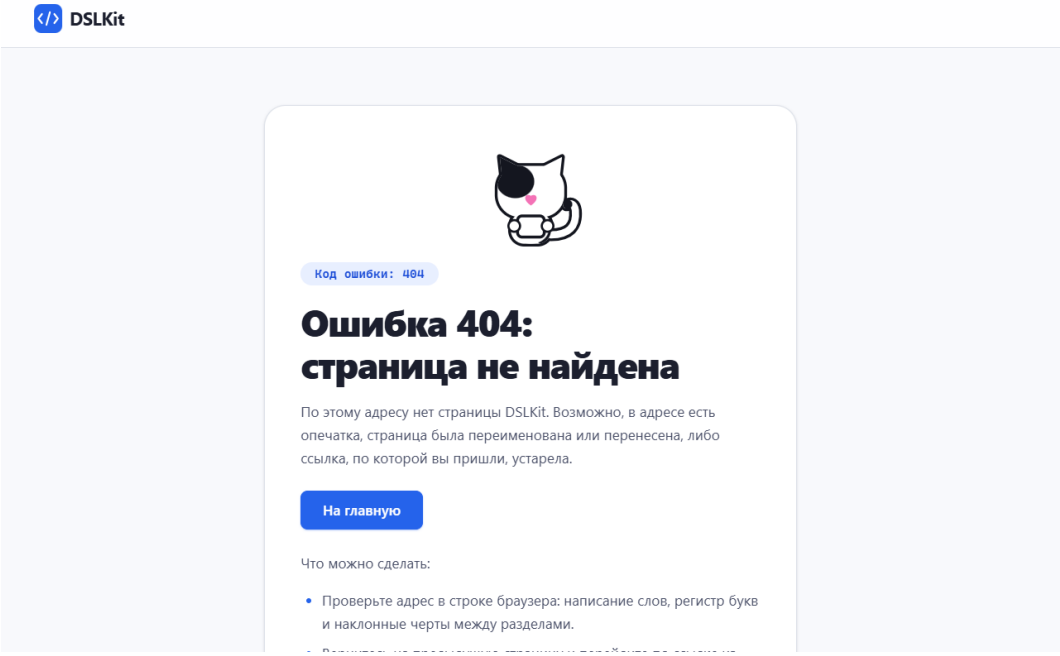


Рис. 5. R5: страница 404

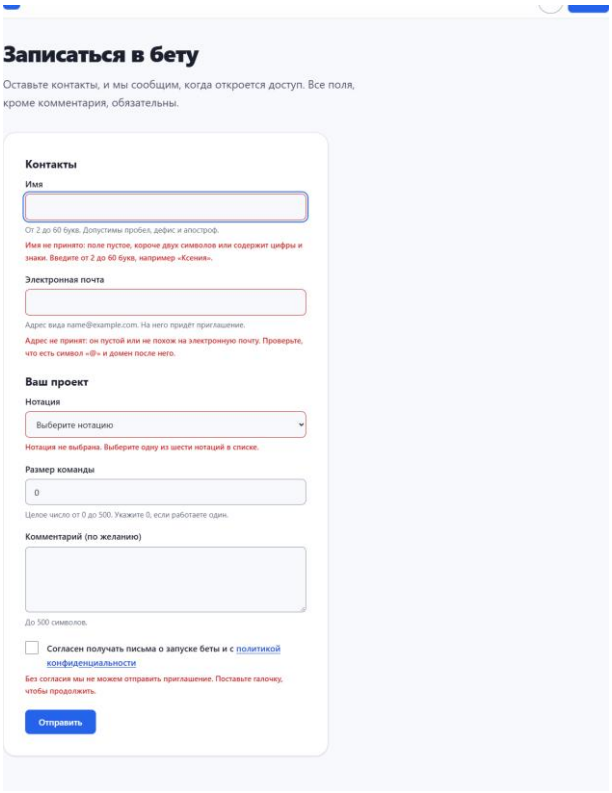


Рис. 6. R4: ошибки валидации формы

```
> node testing-course/lab3/run.js

{
  total: 44,
  passed: 44,
  failed: 0,
  blocked: 0,
  executedPct: 100,
  passedPct: 100,
  defectDensity: 0,
  coveragePct: 100,
  requirements: 9
}
```

Рис. 7. Автоматический прогон чек-листов и метрики

10. Риски и проблемы

Проверки не охватывают визуальную отрисовку диаграмм (внешний сервис Kroki) и другие браузеры. Работа редактора проверена выборочно.

11. Выводы и рекомендации

Выполнено 44 из 44 проверок, дымовой тест пройден на 100%, покрытие требований 100%, дефектов нет. Качество оценено как высокое, релиз рекомендован; браузерные сценарии стоит проверять отдельно.